

동적 공급업체 선정 및 공급량 결정

20181042 김경순

목차

- ◆ 1. 문제의 정의
- ◆ 2. MDP-EI 에피소드 생성
- ◆ 3. AI 아키텍처
- ◆ 4. AI 학습 알고리즘
- ◆ 5. 코드 구현

1. 문제의 정의

- ◆ 다수의 공급업체가 존재하며, 각자 특성이 다른 상황
- ◆ 목적: 발생될 수요를 대응, 재고비용에 맞춰서 모든 비용의 최소화
- ◆ 결정할 행동: 수요충족과 비용최소화를 위한 **공급업체 선택**
및 **구매량 결정**
- ◆ 고려할 사항: 각기 다른 리드타임, 비정상 수요, 업체별 상이한 조건

1. 문제의 정의

◆ 제약 조건

◆ 재고 보존식: 재고 = 이전재고 - 수요 + 도착물량

◆ 용량 제약: 창고 용량 초과 불가

->파이프라인이 길기때문에 단순히 $\text{현재재고} + \text{파이프라인} \leq \text{Cap}$
적용하기 **힘듦**. -> 용량 제약을 '용량페널티 비용'으로써 반영.

1. 문제의 정의

- ◇ 1. 한단위(1일)에서의 영업시간은 08:00시부터 20:00까지이다.
- ◇ 2. 주문은 하루 영업이 시작하기 전에 결정하며, 도착시간(입고시간)은 21:00으로 가정한다.
21:00 도착분은 당일 수요 충족에는 사용할 수 없고, 익일 영업 시작(08:00)부터 사용 가능한 것으로 가정.
- ◇ 3. 수요는 계절성과 트렌드가 반영된 비정상성 수요로 가정한다.(수요 = 기저 수요 $\times$ 월별 가중치 $\times$ 요일별 가중치 + 충격)
- ◇ 4. 특성(MOQ, 배송비, 트럭용량, 개당 비용, 리드타임)이 상이한 5개 업체가 존재한다.
- ◇ 5. 리드타임이 존재하며, 업체마다 다르다. 리드타임은 일정시간을 기준으로 +-기간이 존재하고 확률적으로 결정된다.
주문후 즉시 랜덤리드타임이 확정리드타임으로 바뀐다.
- ◇ 6. 배송비가 업체별로 상이하게 존재하며, 배송비는 트럭 대여료형식으로, 일정수량(=트럭용량)이 넘치기 전까지는 추가비용이 발생하지 않다가 일정 수량이 넘으면 배송비가 추가되는 형식이다.
- ◇ 7.수요를 충족시키지 못했을때, 백오더가 발생하면, 그에 따른 백오더비용(벌칙비용)이 발생하고, 충족시키지못한 수요는 다음날로 이월된다. 백오더 수요를 최우선으로 생각한다. 백오더 비용을 임의로 정한다.
- ◇ 8. 재고보관비용이 존재한다. 재고보관비용을 임의로 정한다.
- ◇ 9. 주문정책은 BSP (업체 C를 기준으로 안전재고3일 산출)를 따라 주문유무와 수량을 결정한다.
- ◇ 10. 주문 수량을 정한 후, 그 수량에 따른 최저가 업체를 선정한다.
- ◇ 11. 창고에는 최대보관용량이 존재한다.
- ◇ 12. 100일간의 스케줄을 최소비용이 들도록 산정한다.

1.문제 정의-변수 설정

일 평균 수요(D): 평균300개 (비정상성 수요)

재고비용 (h): 개당 5원/일

백오더비용(b): 개당 1000원/일

페널티비용(w): 개당 500원/일 (오면 버린다고 생각)

초기 재고: 300개

Table 1: 공급업체별 매개변수 설정 (Supplier Parameters)

ID	Name	Lead Time (τ)	Price (P)	MOQ	Capacity (C_{cap})	Shipping (C_{ship})
1	A	7	80	1000	2000	5000
2	B	5	100	500	1000	3000
3	C	3	120	200	1000	2000
4	D	1	150	50	500	5000
5	E	0	200	0	100	2000

1.1 집합 및 인덱스 (Sets and Indices)

- $t \in \{0, 1, \dots, T\}$: 계획 기간 (일 단위, Day).
- $k \in \mathcal{K} = \{1, \dots, K\}$: 공급업체 집합.

1.2 파라미터 (Parameters)

- τ_k : 공급업체 k 의 확정 리드타임 (Lead time). ($\tau_{\max} = \max_k \tau_k$)
- P_k : 공급업체 k 의 단위 구매 비용.
- MOQ_k : 공급업체 k 의 최소 주문 수량 (Minimum Order Quantity).
- $C_{\text{ship},k}$: 공급업체 k 의 트럭당 운송 비용.
- $C_{\text{cap},k}$: 공급업체 k 의 트럭 1대당 적재 용량.
- h : 단위 기간당 단위 재고 유지 비용 (Holding cost).
- b : 단위 기간당 단위 재고 부족(또는 판매 기회비용) 페널티 (Penalty cost).
- ω : 창고 용량 초과에 대한 단위 페널티.
- CAP : 창고의 최대 적재 용량.
- γ : 할인율. ($\gamma = 1$ 고정)

1.3 상태 변수 (State Variables)

시점 t 에서의 상태 $S_t \in \mathbb{R}^{1+\tau_{\max}+2}$ 는 Zipkin(2008)의 파이프라인 개념을 기반으로 다음과 같이 정의된다.

$$S_t = [I_t, x_t^1, x_t^2, \dots, x_t^{\tau_{\max}}, \sin(\frac{2\pi t}{365}), \cos(\frac{2\pi t}{365})] \quad (1)$$

- I_t : 시점 t 초의 보유 재고 수준.
- x_t^i : 현재 시점(t)으로부터 i 일 후에 도착할 예정인 파이프라인 재고량.
- 마지막 두 항: 계절성 반영을 위한 순환 시간 인코딩.

1.4 의사결정 변수 및 행동 벡터 (Action Vector)

에이전트는 매 시점 t 마다 공급업체 선택(k_t)과 주문량(q_t)을 결정한다.

- $k_t \in \{0, 1, \dots, K\}$: 선택된 공급업체 인덱스 (0이면 주문 안 함).
- $q_t \geq 0$: 선택된 공급업체 k_t 에게 발주할 주문 수량.

이를 통합하여 행동 벡터 A_t 는 다음과 같이 정의된다.

$$A_t = [k_t, q_t] \quad (2)$$

1.5 수요 변수 (Demand Variables)

본 시스템의 수요 D_t 는 고정된 기저 수요에 월별(Monthly) 및 요일별(Day-of-Week) 계절성 계수가 곱해지는 승법 모델로 정의된다.

$$D_t = \max(0, \mu_t + \epsilon_t) \quad (3)$$

- 결정론적 평균 수요 (μ_t): 기저 수요(Base)에 시점 t 가 속한 월(Month)과 요일(Day)의 가중치를 각각 곱하여 산출한다.

$$[\text{평균 수요} = \text{기저 수요} \times \text{월별 가중치} \times \text{요일별 가중치}]$$

$$\mu_t = \text{Base} \times \alpha_{m(t)} \times \beta_{d(t)} \quad (4)$$

여기서 각 항의 정의는 다음과 같다.

- $m(t) \in \{1, \dots, 12\}$: 시점 t 의 월 인덱스.
- $d(t) \in \{0, \dots, 6\}$: 시점 t 의 요일 인덱스.
- $\alpha \in \mathbb{R}^{12}$: 월별 수요 가중치 벡터.
 - * 예: $\alpha = [1.2, 1.1, 1.0, 0.9, 1.0, 1.1, \mathbf{1.5}, \mathbf{1.5}, 1.0, \mathbf{0.8}, \mathbf{0.8}, 1.3]$
- $\beta \in \mathbb{R}^7$: 요일별 수요 가중치 벡터.
 - * 예: $\beta = [1.3, 1.1, 1.0, 1.0, 0.9, 0.5, 0.2]$

- 확률적 간헐적 충격 (ϵ_t):

$$\epsilon_t = \delta_t \cdot Z_t \quad (\delta_t \sim \text{Bernoulli}, Z_t \sim \mathcal{N}) \quad (5)$$

1.6 비용 벡터 (Cost Vector)

시점 t 에서 발생하는 비용을 세부 항목별로 분석하기 위해 비용 벡터 $\mathbf{C}_t \in \mathbb{R}^5$ 를 다음과 같이 정의한다.

$$\mathbf{C}_t = \begin{bmatrix} C_t^{\text{purch}} \\ C_t^{\text{ship}} \\ C_t^{\text{hold}} \\ C_t^{\text{short}} \\ C_t^{\text{cover}} \end{bmatrix} = \begin{bmatrix} P_{k_t} \cdot q_t \cdot \mathbb{I}(k_t > 0) \\ C_{\text{ship}, k_t} \left\lceil \frac{q_t}{C_{\text{cap}, k_t}} \right\rceil \cdot \mathbb{I}(k_t > 0) \\ h \cdot \max(0, I_{t+1}) \\ b \cdot \max(0, -I_{t+1}) \\ \omega \cdot \max(0, I_{t+1} - CAP) \end{bmatrix} \quad (6)$$

각 성분은 순서대로 구매, 운송, 재고 유지, 재고 부족, 용량 초과 비용을 의미한다.

1.7 시스템 동역학 (System Dynamics)

본 시스템의 전이 함수(Transition Function) f 는 현재 상태 S_t , 결정된 행동 $A_t = [k_t, q_t]$, 그리고 실현된 수요 D_t 를 입력으로 받아 다음 시점의 상태 S_{t+1} 을 결정한다.

$$S_{t+1} = f(S_t, A_t, D_t) \quad (7)$$

상태 변수의 각 성분별 전이 규칙은 다음과 같다.

1. **재고 전이 (Inventory Transition):** 시점 t 의 재고 I_t 는 실현된 수요 D_t 만큼 감소하고, 리드타임이 경과하여 시점 t 에 도착한 파이프라인 재고 x_t^1 만큼 증가한다.

$$I_{t+1} = I_t - D_t + x_t^1 \quad (8)$$

2. **파이프라인 전이 (Pipeline Transition):** 파이프라인 벡터는 시간이 흐름에 따라 한 단계 씩 앞으로 이동하며(Shift), 새로 발생한 주문량 q_t 는 해당 공급업체의 리드타임 τ_{k_t} 위치에 삽입된다.

$$x_{t+1}^i = \begin{cases} x_t^{i+1} + q_t \cdot \mathbb{I}(\tau_{k_t} = i) & \text{if } 1 \leq i < \tau_{\max} \\ \mathbb{I}(\tau_{k_t} = \tau_{\max}) \cdot q_t & \text{if } i = \tau_{\max} \end{cases} \quad (9)$$

3. **시간 변수 전이 (Time Feature Transition):** 시간 인코딩 변수는 매 시점 $t \rightarrow t + 1$ 로 선형적으로 증가하며 순환한다.

1.8 목적 함수 (Objective Function)

최종 목표는 계획 기간 T 동안의 총 비용(Total Cost)의 기댓값을 최소화하는 것이다. 총 비용은 비용 벡터의 모든 성분의 합으로 계산된다.

$$\min_{\pi} \mathbb{E} \left[\sum_{t=0}^T \sum_{j=1}^5 \mathbf{C}_t^{(j)} \right] \quad (10)$$

여기서 $\mathbf{C}_t^{(j)}$ 는 비용 벡터 $\mathbf{C}_t$ 의 j 번째 요소를 의미한다.

2. MDP-EI 에피소드 생성

◇ Lecture Note1에서 사용한 ‘Zipkin’의 이론을 빌림

복잡한 리드타임 문제를 벡터로 깔끔하게 풀었다.(차원=Mu+1)

$$S_t = \left[I_t, x_t^1, x_t^2, \dots, x_t^7, \sin\left(\frac{2\pi t}{365}\right), \cos\left(\frac{2\pi t}{365}\right) \right]$$

$$A_t = [k, q]$$

$$\xi_t = D_t$$

$$C_t = [c_{purch}, c_{ship}, c_{hold}, c_{short}, c_{over}]$$

2. MDP-EI 에피소드 생성

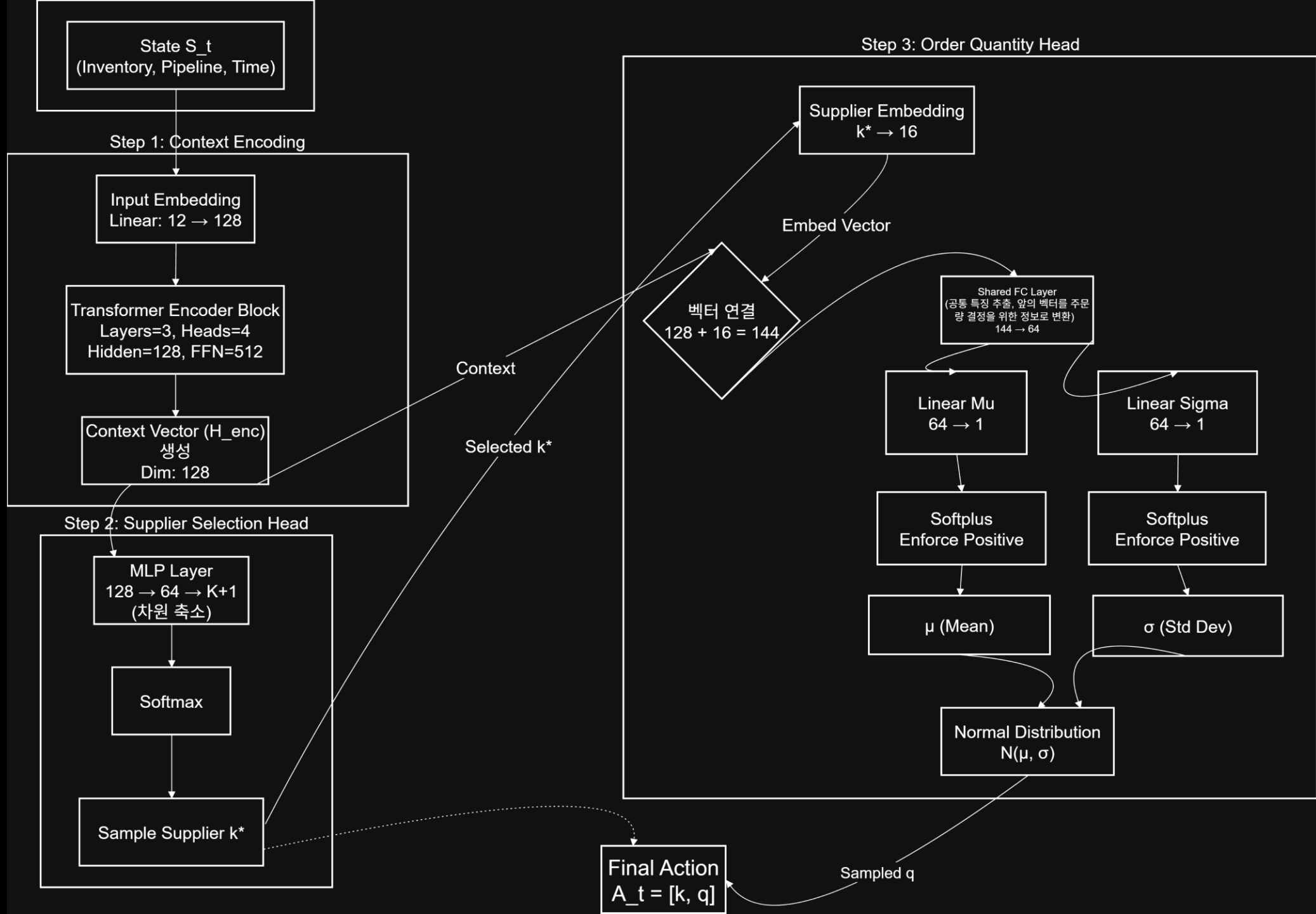
Table 1: Heuristic Policy 기반 재고 관리 시뮬레이션 궤적 (7일)

Time	$S_t = [I_t, x_t^1, x_t^2, \dots, x_t^7, \sin(\frac{2\pi t}{365}), \cos(\frac{2\pi t}{365})]$	$A_t = [k, q]$	D_t	$C_t = [c_{purch}, c_{ship}, c_{hold}, c_{short}, c_{cover}]$
1	[300, 0, 0, 0, 0, 0, 0, 0, 0.2, -1.0]	[1, 2209]	458	[176720, 10000, 0, 158000, 0]
2	[-158, 0, 0, 0, 0, 0, 2209.2, 0, 0.2, -1.0]	[1, 1000]	287	[80000, 5000, 0, 445000, 0]
3	[-445, 0, 0, 0, 0, 2209.2, 1000, 0, 0.2, -1.0]	[0, 0]	278	[0, 0, 0, 723000, 0]
4	[-723, 0, 0, 0, 2209.2, 1000, 0, 0, 0.1, -1.0]	[1, 1000]	159	[80000, 5000, 0, 882000, 0]
5	[-882, 0, 0, 2209.2, 1000, 0, 1000, 0, 0.1, -1.0]	[0, 0]	397	[0, 0, 0, 1279000, 0]
6	[-1279, 0, 2209.2, 1000, 0, 1000, 0, 0, 0.1, -1.0]	[0, 0]	333	[0, 0, 0, 1612000, 0]
7	[-1612, 2209.2, 1000, 0, 1000, 0, 0, 0, 0.1, -1.0]	[0, 0]	327	[0, 0, 1351, 0, 0]

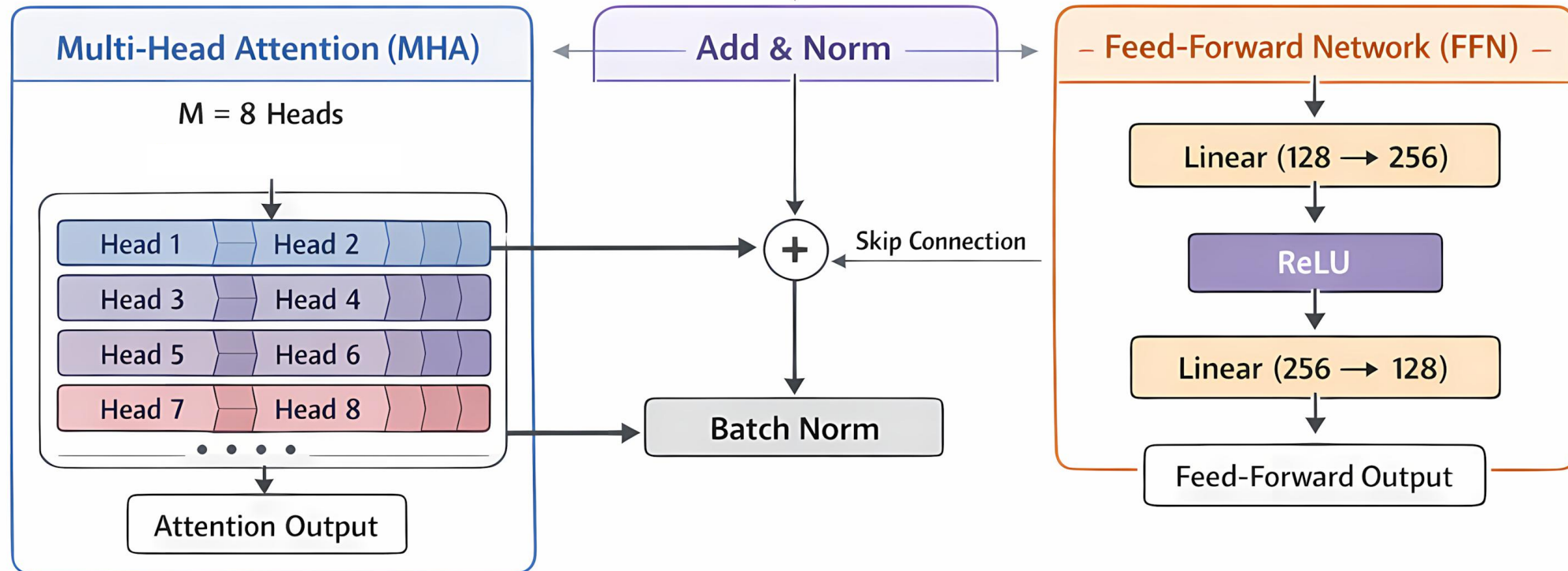
Base-Stock Level (S): 2509 (제일 긴 리드타임 7일기준, 뉴스벤더 분위수를 이용하여 산출)
 이경우엔 먼저 주문량을 결정하고 가장 가성비가 좋은 업체를 선정해 주문함.
 -> 개당 단가가 가장 싼 A만을 고정적으로 주문하게 됨.

3. 모델 아키텍처

- ◆ Lecture Note 3에서 배운 Attention 연산 모델(Transformer)
+Greedy Rollout Baseline을 활용한 모델 설계
- ◆ Attention 연산을 활용하는것은 모델이 재고와 파이프라인, 시간끼리의 Global Context(전체적인 맥락)를 파악하면서 Query와 Key를 통한 연산으로 각 변수마다 중요도를 다르게 보는 Attention 가중치를 적용하게 됨.
-ex.7일뒤 들어오는것보다 1일뒤 들어오는것을 현재 재고입장에서 더 중요하게 봄
- ◆ 행동이 2가지로 분해되었고, 순차적으로 결정하는게 행동공간 최적화에 유리함으로,
 1. 공급업체 선택
 - 2.주문량 결정순으로 디코더에서 순서대로 결정



Transformer Layer



4. 모델 알고리즘

Algorithm 1 다중 공급업체 환경의 인코더-듀얼 디코더 순전파

Require: 상태 S_t (재고, 파이프라인 x_t , 계절성 정보), 파라미터 θ

Ensure: 행동 $C_t = [u_{t,k}, q_{t,k}]$, 행동 확률 $p_\theta(C_t|S_t)$

```
1: 1. 인코더 단계 (State Encoding)
2:    $H_{enc} \leftarrow \text{Encoder}(S_t; \theta_{enc})$  ▷ 상태  $S_t$ 의 특징 벡터 추출
3: 2. 듀얼 디코더 단계 (Dual Decoder Step)
4:   Phase 1: 공급업체 선택 (Supplier Selection)
5:     ▷  $K$ 개의 공급업체 + '주문 안 함(Null)'을 포함한  $K + 1$  차원 로짓
6:      $h_{dec}^{(1)} \leftarrow \text{DecoderBlock}_1(H_{enc}; \theta_{dec1})$ 
7:      $\pi(k|S_t) \leftarrow \text{Softmax}(\text{Linear}(h_{dec}^{(1)}))$ 
8:      $k^* \sim \pi(\cdot|S_t)$  ▷ 확률에 따라 공급업체 인덱스  $k^*$  샘플링
9:     선택된  $k^*$ 에 따라 이진 변수 설정
10:    if  $k^* \in \mathcal{K}$  then
11:       $u_{t,k^*} \leftarrow 1, \quad u_{t,k \neq k^*} \leftarrow 0$ 
12:    else ▷ 주문 안 함 선택 시
13:       $u_{t,k} \leftarrow 0 \quad \forall k$ 
14:    end if
15:    Phase 2: 주문 수량 결정 (Order Quantity)
16:    if  $\sum_k u_{t,k} = 1$  then ▷ 특정 공급업체  $k^*$ 가 선택된 경우
17:       $h_{dec}^{(2)} \leftarrow \text{DecoderBlock}_2(H_{enc}, u_{t,k^*}; \theta_{dec2})$ 
18:      정책 분포 파라미터 도출 (최적 주문량에 대한 평균  $\mu$ , 표준편차  $\sigma$ )
19:       $\mu_t, \sigma_t \leftarrow \text{ProjectionHead}(h_{dec}^{(2)})$ 
20:       $q_{t,k^*} \sim \mathcal{N}(\mu_t, \sigma_t)$  ▷ 정규분포  $\mathcal{N}$ 에서 수량 샘플링
21:       $q_{t,k \neq k^*} \leftarrow 0$ 
22:    else
23:       $q_{t,k} \leftarrow 0 \quad \forall k$ 
24:    end if
25: 3. 최종 행동 및 확률 반환
26:    $C_t \leftarrow [u_{t,k}, q_{t,k}]$ 
27:    $p_\theta(C_t|S_t) \leftarrow \pi(k^*|S_t) \cdot p(q_{t,k^*}|k^*, S_t)$ 
28: return  $C_t, p_\theta(C_t|S_t)$ 
```

4. 모델 알고리즘

Algorithm 2 Greedy Rollout Baseline을 활용한 강화 학습

Require: 계획 기간 T , 에폭 E , 배치 B , 공급업체 집합 $\mathcal{K}$

```

1: 정책 네트워크  $\theta$  초기화, 베이스라인  $\theta^{BL} \leftarrow \theta$ 
2: for epoch = 1, ...,  $E$  do
3:   for batch  $i = 1, \dots, B$  do
4:     초기 상태  $S_0$  생성
5:     1. 정책 비교 (Advantage 계산)
6:       a) 샘플링 롤아웃 (Exploration)
7:        $C_{0:T}^{sample}, \text{Cost}^{sample} \leftarrow \text{RunEpisode}(\theta, S_0, \text{mode}='Sample')$ 
8:       b) 그리디 롤아웃 (Baseline)
9:        $C_{0:T}^{greedy}, \text{Cost}^{greedy} \leftarrow \text{RunEpisode}(\theta^{BL}, S_0, \text{mode}='Greedy')$ 
10:    2. 그래디언트 계산
11:     $\text{Advantage} \leftarrow (\text{Cost}^{sample} - \text{Cost}^{greedy})$ 
12:     $\nabla J \leftarrow \frac{1}{B} \sum_{t=0}^T \text{Advantage} \cdot \nabla_{\theta} \log p_{\theta}(C_t^{sample} | S_t)$ 
13:     $\theta \leftarrow \text{Adam}(\theta, \nabla J)$ 
14:   end for
15:   3. 베이스라인 업데이트 검증 (t-test)
16:   if Validate( $\theta$ ) is significantly better than Validate( $\theta^{BL}$ ) then
17:      $\theta^{BL} \leftarrow \theta$  ▷ 베이스라인 갱신
18:   end if
19: end for

20: function RUNEPISODE( $\theta, S_0, \text{mode}$ )
21:   Total Cost  $L \leftarrow 0$ 
22:   for  $t = 0, \dots, T$  do
23:      $C_{t,-} \leftarrow \text{Forward}(S_t; \theta, \text{mode})$  ▷ Alg 1 호출
24:     비용 계산 (구매 + 운송 + 재고 + 페널티)
25:      $\text{Cost}_t = \sum_{k \in \mathcal{K}} (P_k q_{t,k} + C_{\text{ship},k} \lceil \frac{q_{t,k}}{C_{\text{cap},k}} \rceil)$ 
26:        $+ h[I_{t+1}]^+ + b[I_{t+1}]^- + \omega[I_{t+1} - CAP]^+$ 
27:      $L \leftarrow L + \text{Cost}_t$ 
28:      $S_{t+1} \leftarrow \text{StateTransition}(S_t, C_t)$  ▷ 식 (2), (3) 적용
29:   end for
30:   return  $C_{0:T}, L$ 
31: end function

```

5. 코드 구현

```
# =====
# 1. Configuration (통합)
# =====
CONFIG = {
    'h': 5.0,          # 단위당 보관비 (Holding Cost)
    'b': 1000.0,       # 단위당 품질비 (Backorder Cost)
    'omega': 500.0,    # 초과 보관비 (Over-capacity Penalty) [상향 조정됨]
    'CAP': 2000.0,     # 창고 용량
    'base_demand': 300.0, # 기저 수요 (일평균)
    'days': 60,       # 에피소드 길이
    'tau_max': 7,      # 최대 파이프라인 길이
    'gamma': 1.0,      # 할인율
}

TRAIN_CONFIG = {
    'lr': 1e-4,
    'batch_size': 512,
    'epochs': 50,
    'steps_per_epoch': 100, # [핵심] 에폭당 업데이트 횟수
    'val_episodes': 5000,  # [핵심] 검증 데이터 수
    'hidden_dim': 128,
    'supplier_emb_dim': 32,
    'num_layers': 3,       # 모델 깊이
    'num_heads': 8         # 헤드 수
}

SUPPLIERS = {
    0: {'name': 'None', 'tau': 0, 'P': 0, 'MOQ': 0, 'C_cap': 1, 'C_ship': 0},
    1: {'name': 'A', 'tau': 7, 'P': 80, 'MOQ': 1000, 'C_cap': 2000, 'C_ship': 5000},
    2: {'name': 'B', 'tau': 5, 'P': 100, 'MOQ': 500, 'C_cap': 1000, 'C_ship': 3000},
    3: {'name': 'C', 'tau': 3, 'P': 120, 'MOQ': 200, 'C_cap': 1000, 'C_ship': 2000},
    4: {'name': 'D', 'tau': 1, 'P': 150, 'MOQ': 50, 'C_cap': 500, 'C_ship': 5000},
    5: {'name': 'E', 'tau': 0, 'P': 200, 'MOQ': 0, 'C_cap': 100, 'C_ship': 2000}
}
```

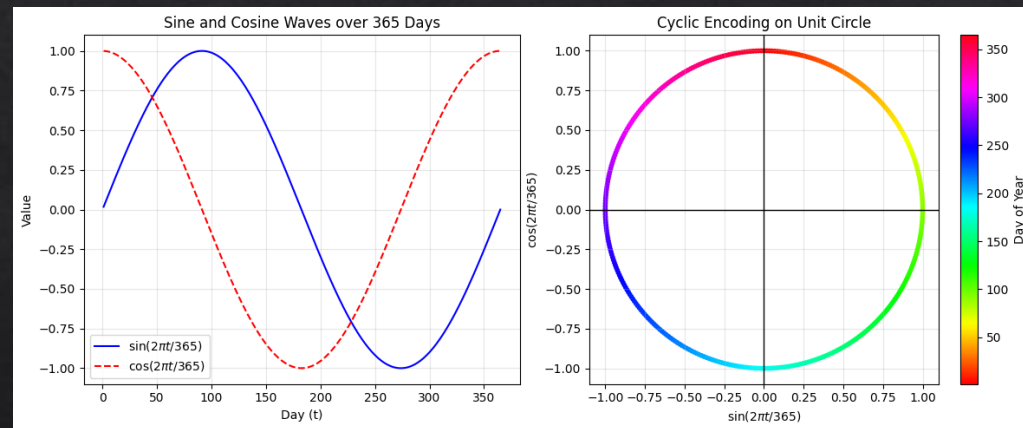
환경과 파라미터 정의

5. 코드 구현

```
# =====  
# 2. Environment  
# =====  
class VectorizedInventoryEnv:  
    def __init__(self, config, batch_size):  
        self.cfg = config  
        self.batch_size = batch_size  
        self.tau_max = config['tau_max']  
        self.seasonal_month = torch.tensor([1.0, 1.0, 0.9, 1.0, 1.1, 1.2, 1.5, 1.5, 1.0, 0.9, 1.1, 1.3], device=DEVICE)  
        self.seasonal_day = torch.tensor([1.2, 1.1, 1.0, 1.0, 0.9, 0.6, 0.5], device=DEVICE)  
  
    def reset(self):  
        self.on_hand = torch.rand(self.batch_size).to(DEVICE) * self.cfg['CAP']  
        self.pipeline = torch.zeros(self.batch_size, self.tau_max + 1).to(DEVICE)  
        start_day = torch.randint(0, 365 - self.cfg['days'], (1,)).item()  
        self.t_global = start_day  
        self.t_step = 0  
        return self._get_state()  
  
    def _get_state(self):  
        # S_t 구성 요소  
        inv_feat = self.on_hand.unsqueeze(1) / self.cfg['CAP']  
        pipe_feat = self.pipeline[:, 1:] / self.cfg['CAP']  
        sin_t = math.sin(2 * math.pi * self.t_global / 365)  
        cos_t = math.cos(2 * math.pi * self.t_global / 365)  
        time_feat = torch.tensor([[sin_t, cos_t]], device=DEVICE).repeat(self.batch_size, 1)  
        state = torch.cat([inv_feat, pipe_feat, time_feat], dim=1)  
        return state  
  
    def _get_demand(self):  
        current_month = (self.t_global // 30) % 12  
        current_day = self.t_global % 7  
        factor_m = self.seasonal_month[current_month]  
        factor_d = self.seasonal_day[current_day]  
        mu_t = self.cfg['base_demand'] * factor_m * factor_d  
        sigma_t = mu_t * 0.2  
        demand = torch.normal(mean=mu_t.float(), std=sigma_t.float(), size=(self.batch_size,)).to(DEVICE)
```

비정상 수요 구현

State_t에 요일정보를 반영
->순환 특징 인코딩



```

# =====
# 3. Model Architecture
# =====
class InventoryTransformer(nn.Module):
    def __init__(self, state_dim, hidden_dim, num_layers=2, num_heads=4):
        super(InventoryTransformer, self).__init__()

        self.input_embed = nn.Linear(state_dim, hidden_dim)
        encoder_layer = nn.TransformerEncoderLayer(d_model=hidden_dim, nhead=num_heads, dim_feedforward=256, batch_first=True)
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        self.supplier_head = nn.Sequential(
            nn.Linear(hidden_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 6)
        )

        self.sup_action_embed = nn.Embedding(6, TRAIN_CONFIG['supplier_emb_dim'])

        self.qty_input_dim = hidden_dim + TRAIN_CONFIG['supplier_emb_dim']
        self.qty_head_mu = nn.Sequential(
            nn.Linear(self.qty_input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
        )

        self.qty_head_sigma = nn.Sequential(
            nn.Linear(self.qty_input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
            nn.Softplus()
        )

        # Bias 유지 (초기 소량 주문 유도)
        self.qty_head_mu[2].bias.data.fill_(5.0)

```

입력 상태 S_t 를
128차원의
특징 벡터로 변환

Transformer 신경망을
쓰으로써 Self- Attention

MLP Layer
(128 $\rightarrow$ 64 $\rightarrow$ 6)

자기회귀:
방금 선택된 공급업체의
정보를 다시 32차원의
벡터로 임베딩

주문량의 평균(μ)과
표준편차(σ)를 산출

```

# =====
# 4. Trainer
# =====
class RolloutReinforceTrainer:
def run_episode(self, target_model, deterministic=False):
    state = self.env.reset()
    current_batch_size = self.env.batch_size
    total_rewards = torch.zeros(current_batch_size).to(DEVICE)
    log_probs = []

    while True:
        sup_idx, qty, lp_s, lp_q = target_model.act(state, deterministic)
        next_state, reward, done, info = self.env.step(sup_idx, qty)

        scaled_reward = reward * 1e-5
        total_rewards += scaled_reward

        if not deterministic:
            log_probs.append(lp_s + lp_q)

        state = next_state
        if done: break

    return total_rewards, log_probs

def train_epoch(self, epoch_idx):
    self.model.train()
    avg_reward = 0
    avg_baseline = 0

    for _ in range(self.steps):
        rewards, log_probs_list = self.run_episode(self.model, deterministic=False)
        log_probs = torch.stack(log_probs_list).sum(dim=0)

        with torch.no_grad():
            baseline_rewards, _ = self.run_episode(self.baseline_model, deterministic=True)

        advantage = rewards - baseline_rewards
        pg_loss = -(advantage * log_probs).mean()

        self.optimizer.zero_grad()
        pg_loss.backward()
        torch.nn.utils.clip_grad_norm_(self.model.parameters(), max_norm=1.0)
        self.optimizer.step()

        avg_reward += rewards.mean().item()
        avg_baseline += baseline_rewards.mean().item()

    self.scheduler.step()
    return -(avg_reward / self.steps), -(avg_baseline / self.steps)

```

에피소드 시뮬레이션

-에이전트가 환경과 상호작용하며 데이터를 수집
-에이전트의 정책에 따라 업체와 주문량 결정
->보상을받고 다음상태로 전이

비용을 작은 수치로 스케일링하여 안정적인 기울기

baseline_model을 사용하여 Deterministic하게 에피소드를 실행

rewards - baseline_rewards로 Baseline 교체의 기준점을 잡음 -> p-value 계산

손실함수 $pg_loss = -(advantage * log_probs).mean()$:
좋은 행동을 더 하도록 모델을 업데이트함(확률을 높임)

기울기 클리핑, 스케줄러

```

def update_baseline_if_better(self, epoch):
    val_env = VectorizedInventoryEnv(CONFIG, batch_size=TRAIN_CONFIG['val_episodes'])
    original_env = self.env
    self.env = val_env

    with torch.no_grad():
        curr_rewards, _ = self.run_episode(self.model, deterministic=True)
        base_rewards, _ = self.run_episode(self.baseline_model, deterministic=True)

    self.env = original_env

    curr_costs = -curr_rewards.cpu().numpy()
    base_costs = -base_rewards.cpu().numpy()

    t_stat, p_val = ttest_rel(curr_costs, base_costs)
    mean_curr = curr_costs.mean()
    mean_base = base_costs.mean()

    improvement_pct = (mean_base - mean_curr) / mean_base * 100
    is_improved = (mean_base > mean_curr) and (p_val < 0.05) #Kool et al. (2019)논문을 따름

    status_icon = "✅ UPDATED" if is_improved else "🚫 KEPT"
    diff_str = f"📉 -{improvement_pct:5.2f}%" if improvement_pct > 0 else f"📈 +{-improvement_pct:5.2f}%"
    print(f" >>> {status_icon} | Val: {mean_curr:9.1f} vs Base: {mean_base:9.1f} | {diff_str} (p={p_val:.3f})")

    if is_improved:
        self.baseline_model.load_state_dict(self.model.state_dict())
        if os.path.exists(SAVE_DIR):
            save_path = os.path.join(SAVE_DIR, f"best_model_epoch_{epoch:03d}_cost_{int(mean_curr)}.pt")
            torch.save(self.model.state_dict(), save_path)
            print(f" 📁 Saved to Drive: {save_path}")

```

검증 에피소드 실행
val_episodes=5000

Paired t-test 수행

BL 교체 조건($p_val < 0.05$)


```
def _get_demand(self):
    current_month = (self.t_global // 30) % 12
    current_day = self.t_global % 7
    factor_m = self.seasonal_month[current_month]
    factor_d = self.seasonal_day[current_day]
    mu_t = self.cfg['base_demand'] * factor_m * factor_d
    sigma_t = mu_t * 0.2
    demand = torch.normal(mean=mu_t.float(), std=sigma_t.float(), size=(self.batch_size,)).to(DEVICE)
    return torch.clamp(demand, min=0)
```

월별/요일별 계절성 지수 적용
수요생성, round로 정수화

```
def step(self, supplier_idx, order_qty):
    cost_purch = torch.zeros(self.batch_size).to(DEVICE)
    cost_ship = torch.zeros(self.batch_size).to(DEVICE)
    instant_arrival = torch.zeros(self.batch_size).to(DEVICE)
```

MOQ보다 작으면 강제로 상향 보정, 차량
용량에 따른 트럭 대수, 배송비 계산

```
for s_code, s_info in SUPPLIERS.items():
    if s_code == 0: continue
    mask = (supplier_idx == s_code)
    if mask.sum() == 0: continue
```

```
raw_qty = order_qty[mask]
corrected_qty = torch.max(raw_qty, torch.tensor(float(s_info['MOQ']), device=DEVICE))
order_qty[mask] = corrected_qty
```

```
tau = s_info['tau']
if tau == 0:
    instant_arrival[mask] += corrected_qty
else:
    if tau <= self.tau_max:
        self.pipeline[mask, tau] += corrected_qty
```

```
cost_purch[mask] += corrected_qty * s_info['P']
trucks = torch.ceil(corrected_qty / s_info['C_cap'])
cost_ship[mask] += trucks * s_info['C_ship']
```

```
arrived = self.pipeline[:, 1].clone() + instant_arrival
self.pipeline[:, :-1] = self.pipeline[:, 1:].clone()
self.pipeline[:, -1] = 0
self.on_hand += arrived
```

pipeline 행렬에 주문량을
기록하고, 하루가 지날 때마다
한 칸씩 재고 쪽으로 이동하는
시프트 연산 수행

```
# 수요 발생
demands = torch.normal(mean=self.cfg['d_mean'], std=self.cfg['d_std'], size=(self.batch_size,)).to(DEVICE)
demands = torch.clamp(demands, min=0).round() # 소수점 반올림하여 정수형태의 실수로 변환
```

```
# 비용 계산
cost_hold = self.cfg['h'] * torch.relu(self.on_hand)
cost_short = self.cfg['b'] * torch.relu(-self.on_hand)
cost_over = self.cfg['omega'] * torch.relu(self.on_hand - self.cfg['CAP'])
```

보관비용, 백오더비용, 초과 페널티비용
계산

```
total_cost = cost_purch + cost_ship + cost_hold + cost_short + cost_over
reward = -total_cost
```

```
self.t_global += 1
self.t_step += 1
done = (self.t_step >= self.cfg['days'])
```

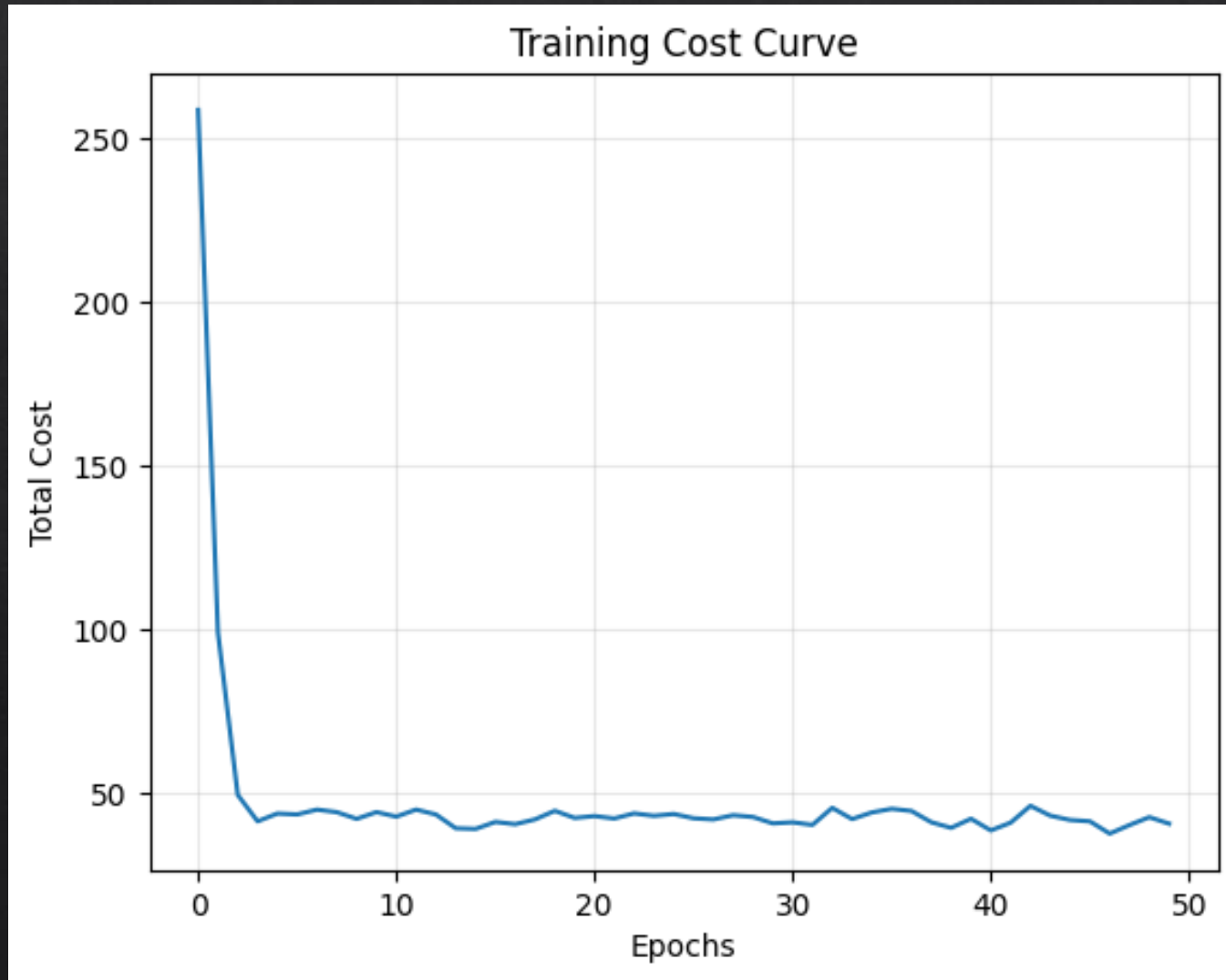
```
info = {
    'total_cost': total_cost,
    'demand': demands,
    # C_t 벡터 구성 요소 [Purch, Ship, Hold, Short, Over]
    'cost_vec': torch.stack([cost_purch, cost_ship, cost_hold, cost_short, cost_over], dim=1)
}
```

```
return self._get_state(), reward, done, info
```

```
Using Device: cuda
Training Started... (Steps/Epoch: 100, Batch: 512)
Models will be saved to: /content/drive/MyDrive/inventory_models
Epoch 001 | Train Cost: 258.5 | >>> [x] UPDATED | Val: 144.6 vs Base: 836.9 | [x] -82.72% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_001_cost_144.pt
Epoch 002 | Train Cost: 99.1 | >>> [x] UPDATED | Val: 31.1 vs Base: 104.1 | [x] -70.11% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_002_cost_31.pt
Epoch 003 | Train Cost: 49.5 | >>> [x] UPDATED | Val: 34.9 vs Base: 67.1 | [x] -47.94% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_003_cost_34.pt
Epoch 004 | Train Cost: 41.5 | >>> [x] KEPT | Val: 28.0 vs Base: 27.7 | [x] + 0.99% (p=0.258)
Epoch 005 | Train Cost: 43.8 | >>> [x] UPDATED | Val: 29.1 vs Base: 54.4 | [x] -46.51% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_005_cost_29.pt
Epoch 006 | Train Cost: 43.6 | >>> [x] UPDATED | Val: 29.6 vs Base: 31.7 | [x] - 6.38% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_006_cost_29.pt
Epoch 007 | Train Cost: 45.0 | >>> [x] UPDATED | Val: 43.4 vs Base: 61.6 | [x] -29.46% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_007_cost_43.pt
Epoch 008 | Train Cost: 44.3 | >>> [x] KEPT | Val: 40.8 vs Base: 37.3 | [x] + 9.46% (p=0.000)
Epoch 009 | Train Cost: 42.2 | >>> [x] KEPT | Val: 35.8 vs Base: 31.9 | [x] +12.32% (p=0.000)
Epoch 010 | Train Cost: 44.3 | >>> [x] UPDATED | Val: 27.1 vs Base: 28.3 | [x] - 4.55% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_010_cost_27.pt
Epoch 011 | Train Cost: 42.9 | >>> [x] KEPT | Val: 74.8 vs Base: 28.6 | [x] +161.82% (p=0.000)
Epoch 012 | Train Cost: 45.1 | >>> [x] KEPT | Val: 46.9 vs Base: 29.0 | [x] +61.49% (p=0.000)
Epoch 013 | Train Cost: 43.6 | >>> [x] UPDATED | Val: 71.8 vs Base: 77.4 | [x] - 7.15% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_013_cost_71.pt
Epoch 014 | Train Cost: 39.4 | >>> [x] KEPT | Val: 33.7 vs Base: 31.5 | [x] + 6.95% (p=0.000)
Epoch 015 | Train Cost: 39.2 | >>> [x] UPDATED | Val: 32.1 vs Base: 35.3 | [x] - 9.14% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_015_cost_32.pt
Epoch 016 | Train Cost: 41.3 | >>> [x] KEPT | Val: 31.2 vs Base: 29.9 | [x] + 4.38% (p=0.000)
Epoch 017 | Train Cost: 40.6 | >>> [x] KEPT | Val: 31.0 vs Base: 26.7 | [x] +16.23% (p=0.000)
Epoch 018 | Train Cost: 42.1 | >>> [x] KEPT | Val: 53.4 vs Base: 45.1 | [x] +18.51% (p=0.000)
Epoch 019 | Train Cost: 44.7 | >>> [x] KEPT | Val: 28.1 vs Base: 26.9 | [x] + 4.40% (p=0.000)
Epoch 020 | Train Cost: 42.5 | >>> [x] KEPT | Val: 67.1 vs Base: 30.4 | [x] +120.63% (p=0.000)
Epoch 021 | Train Cost: 43.1 | >>> [x] KEPT | Val: 88.0 vs Base: 84.8 | [x] + 3.82% (p=0.012)
Epoch 022 | Train Cost: 42.3 | >>> [x] KEPT | Val: 32.6 vs Base: 30.7 | [x] + 6.17% (p=0.000)
Epoch 023 | Train Cost: 43.9 | >>> [x] KEPT | Val: 28.8 vs Base: 28.5 | [x] + 1.03% (p=0.175)
Epoch 024 | Train Cost: 43.1 | >>> [x] KEPT | Val: 36.2 vs Base: 27.7 | [x] +30.44% (p=0.000)
Epoch 025 | Train Cost: 43.7 | >>> [x] UPDATED | Val: 47.0 vs Base: 60.0 | [x] -21.73% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_025_cost_46.pt
Epoch 026 | Train Cost: 42.4 | >>> [x] KEPT | Val: 28.8 vs Base: 28.2 | [x] + 2.04% (p=0.021)
Epoch 027 | Train Cost: 42.1 | >>> [x] UPDATED | Val: 38.2 vs Base: 39.1 | [x] - 2.23% (p=0.033)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_027_cost_38.pt
Epoch 028 | Train Cost: 43.3 | >>> [x] UPDATED | Val: 31.7 vs Base: 68.8 | [x] -53.87% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_028_cost_31.pt
Epoch 029 | Train Cost: 42.8 | >>> [x] KEPT | Val: 48.7 vs Base: 31.4 | [x] +55.04% (p=0.000)
Epoch 030 | Train Cost: 40.8 | >>> [x] UPDATED | Val: 28.6 vs Base: 39.5 | [x] -27.55% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_030_cost_28.pt
Epoch 031 | Train Cost: 41.2 | >>> [x] UPDATED | Val: 35.3 vs Base: 64.5 | [x] -45.33% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_031_cost_35.pt
Epoch 032 | Train Cost: 40.4 | >>> [x] KEPT | Val: 31.5 vs Base: 31.3 | [x] + 0.68% (p=0.422)
Epoch 033 | Train Cost: 45.6 | >>> [x] KEPT | Val: 30.6 vs Base: 26.7 | [x] +14.62% (p=0.000)
Epoch 034 | Train Cost: 42.2 | >>> [x] KEPT | Val: 99.2 vs Base: 46.6 | [x] +112.70% (p=0.000)
Epoch 035 | Train Cost: 44.2 | >>> [x] UPDATED | Val: 29.1 vs Base: 75.5 | [x] -61.46% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_035_cost_29.pt
Epoch 036 | Train Cost: 45.3 | >>> [x] KEPT | Val: 113.7 vs Base: 27.7 | [x] +310.18% (p=0.000)
Epoch 037 | Train Cost: 44.6 | >>> [x] UPDATED | Val: 27.7 vs Base: 29.8 | [x] - 6.99% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_037_cost_27.pt
Epoch 038 | Train Cost: 41.2 | >>> [x] KEPT | Val: 30.5 vs Base: 27.2 | [x] +11.85% (p=0.000)
Epoch 039 | Train Cost: 39.5 | >>> [x] KEPT | Val: 92.0 vs Base: 27.0 | [x] +240.77% (p=0.000)
Epoch 040 | Train Cost: 42.3 | >>> [x] UPDATED | Val: 26.7 vs Base: 31.8 | [x] -16.18% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_040_cost_26.pt
Epoch 041 | Train Cost: 38.7 | >>> [x] KEPT | Val: 67.3 vs Base: 28.2 | [x] +138.40% (p=0.000)
Epoch 042 | Train Cost: 41.1 | >>> [x] UPDATED | Val: 30.4 vs Base: 70.7 | [x] -57.06% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_042_cost_30.pt
Epoch 043 | Train Cost: 46.2 | >>> [x] UPDATED | Val: 39.6 vs Base: 59.0 | [x] -32.79% (p=0.000)
  Saved to Drive: /content/drive/MyDrive/inventory_models/best_model_epoch_043_cost_39.pt
```

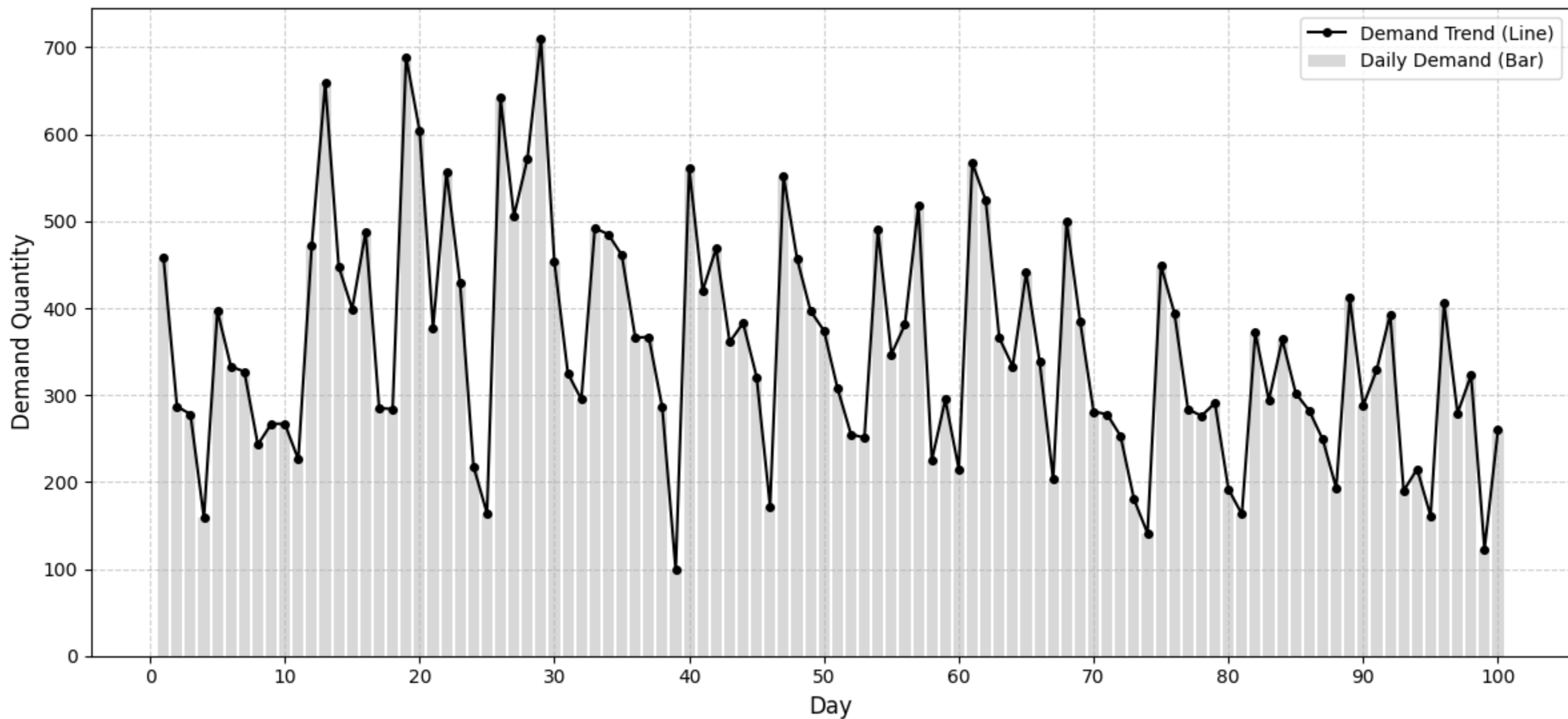
학습중인 모습
로그를 출력하도록해 학습이 잘되고
있는지 알 수 있음

5. 코드 구현 - 학습 그래프



5. 코드 구현-100일간의 시뮬레이션 비교

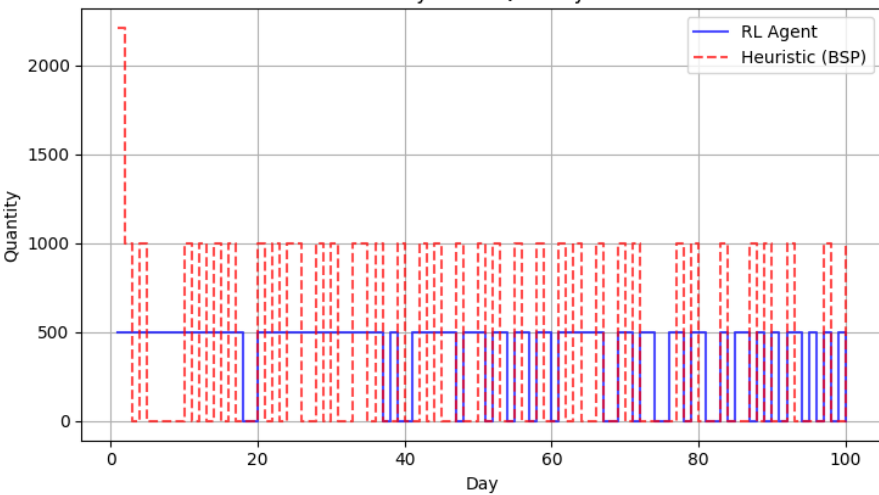
Daily Demand Profile over Simulation Period



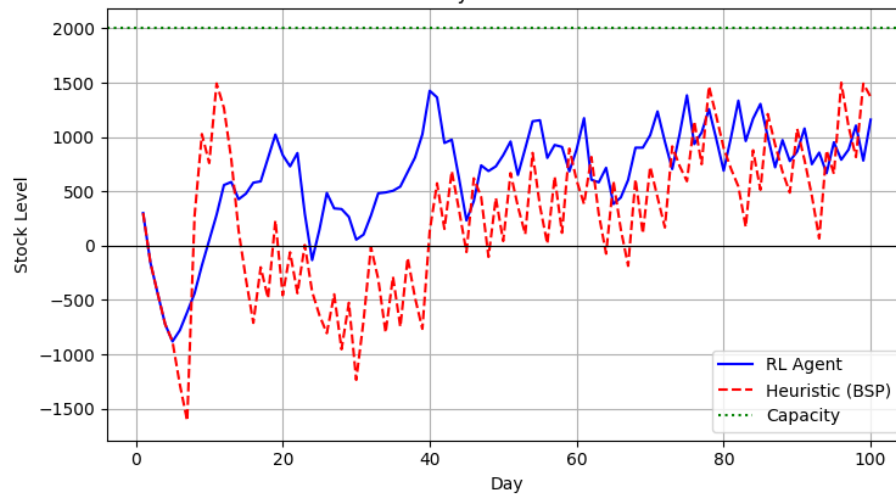
5. 코드 구현-100일간의 시뮬레이션 비교

Inventory Management Policy Comparison: RL vs Heuristic

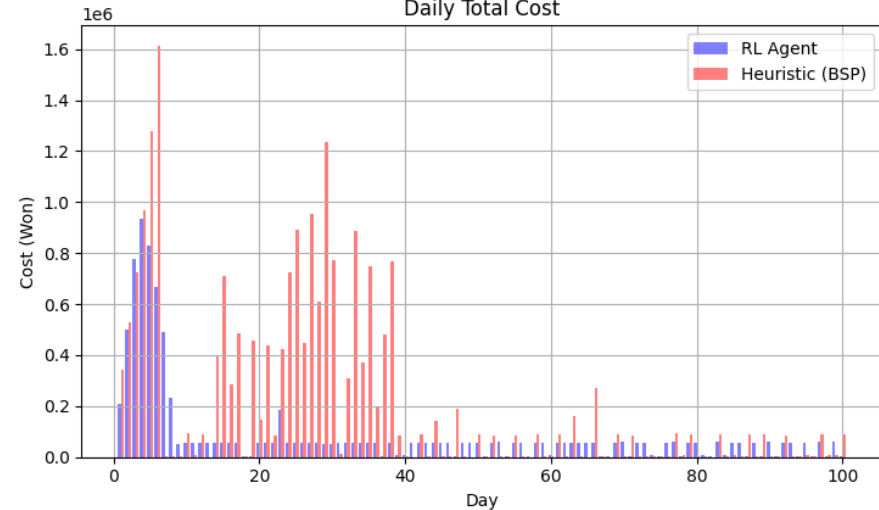
Daily Order Quantity



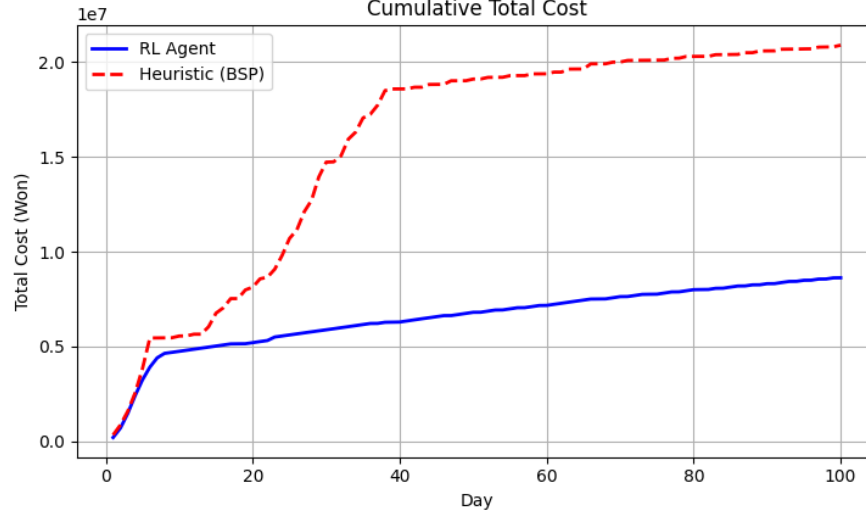
Inventory Level Over Time



Daily Total Cost



Cumulative Total Cost



뉴스벤더 분위기를 따르는 BSP
Total Cost: 20,884,283

VS
Attention with Greedy Rollout
Baseline 모델
Total Cost: 8,626,090

59퍼센트 수준 비용감소